

CS-330 Comp Graphic and Visualization

7-1 Final Project Submission

Malgorzata Debska

Southern New Hampshire University

Dr. Brian Holbert

December 9, 2024

In developing my 3D scene, the primary focus was to create an environment that is both visually immersive and functionally intuitive. This required deliberate thought in object selection, navigation mechanics, camera control, and modularity within the program's structure. These decisions were informed by a combination of user experience design principles, performance optimization considerations, and programming best practices to ensure that the scene was not only aesthetically appealing but also efficient and reusable. The goal was to create an engaging 3D environment that not only showcases basic principles of 3D graphics and game development but also supports intuitive user interaction and scalability for future enhancements.

Justification for Object Selection

For the 3D scene, I chose a cylinder and a cone as the primary objects to demonstrate fundamental 3D graphic principles. The cylinder acts as a base object, representing cylindrical shapes and their texturing, which are common in real-world applications. This choice allowed me to showcase basic geometric shapes and their transformations, such as scaling, rotation, and translation. The cone, on the other hand, adds variety by introducing a conical shape, which allows me to explore lighting and texturing from a unique perspective. This diversity in object selection enabled me to test various aspects of lighting, including how light interacts with different surface geometries and textures. Texturing was a key consideration, with simple textures used to maintain a basic implementation while still providing room for differentiation. These textures were chosen to highlight the texturing technique and the handling of texture coordinates effectively within OpenGL. Lighting was implemented using shaders to simulate realistic conditions in the 3D scene. I used a directional light applied to both the cylinder and cone to demonstrate basic lighting calculations, including shadowing and ambient occlusion. This setup is fundamental in simulating how light behaves in 3D environments, making the scene

visually more realistic and engaging. The camera control system was designed to enable users to navigate the scene effectively, adjusting their view to explore different angles of the objects. This feature is crucial to making the interaction of the user such that they can manipulate their view and observe how changes in the view affect lighting and texture mapping. Camera control utilizes mouse input for rotation and scrolling for zooming, which is a common practice in 3D applications. This choice was guided by the need to provide a user-friendly interface that simulates real-world camera handling in 3D modeling and animation software. Scalability and extensibility were also important considerations in my development choices. The selected objects and functionalities serve as a foundational base that can be expanded upon in future projects. For example, I can easily add more complex objects, use more sophisticated lighting models, or create interactive features that respond to user input. This approach allows me to build a scene that not only meets the current project goals but also highlights basic concepts in 3D graphics, such as geometry, texturing, lighting, and user interaction.

Navigation in My 3D Scene

To navigate the 3D scene, I set up a virtual camera that enables users to interact with the environment in an intuitive manner. The camera control system allows users to move around the scene with greater flexibility. Camera control using mouse and keyboard inputs has been implemented. On mouse input, I integrated a system that can rotate the camera around objects present in the scene. This is achieved by mapping mouse movement to changes in the camera's orientation. As users move the mouse horizontally, the camera pans left and right, showing more of the scene. Vertical mouse movement adjusts the camera's tilt, allowing users to look up or down. This setup mirrors real-world 3D modeling software where users often need to change their viewpoint to inspect objects from different angles. The scroll wheel of the mouse serves as

a tool for zooming in and out. Scrolling forward zooms in, bringing objects closer to the user, which is helpful during detailed inspections. Scrolling backward zooms out, allowing the user to see the entire scene from a broader perspective. This makes the navigation process more dynamic and responsive as users can easily switch between detailed and wide views.

Additionally, keyboard inputs complement the mouse controls to provide a more comprehensive navigation experience. Arrow keys allow users to move the camera forward, backward, left, or right along the X and Z axes. This enables users to navigate in a more straightforward manner, akin to walking through the scene. By combining keyboard and mouse inputs, users have more control over their movement, facilitating smooth exploration of the 3D environment. This setup not only enhances user interaction but also aligns with industry standards, making it easier for users to adapt to the navigation system regardless of their level of experience with 3D graphics software.

The virtual camera is a core component of the scene, responsible for delivering an immersive perspective. I used a first-person camera system that moves in coordination with user inputs, simulating real-world movement and head-turning mechanics. Smooth transitions were employed to ensure that the camera movement appears natural to the user, rather than abrupt. To prevent disorientation, the vertical rotation of the camera was limited to a reasonable range, allowing users to look up and down without flipping the view upside down. Collision detection was integrated into the camera control, ensuring it cannot pass through solid objects like walls or ceilings, which helps maintain a realistic and immersive environment. On top of the standard first-person view, I implemented a free-camera mode. This mode allows users to toggle into a floating, unrestricted camera that can move freely around the scene. It is particularly useful for inspecting objects, exploring the environment from different angles, or for debugging purposes.

during development. The free-camera mode includes adjustable speed controls, enabling users to navigate large spaces quickly or explore trivial details with precision.

Reuse and Modular Code

To enhance modularity and readability, I created several personal functions to handle specific tasks independently of the main program. These functions aim to ensure more maintainability and code reuse for the project. Each function serves a distinct purpose, allowing the main program to focus solely on its essential tasks without redundancy.

- **CameraControl():** This method manages the virtual camera control, including all user inputs from mouse and keyboard to rotate the orientation, position, and zoom of the camera. It centralizes all logic concerning camera movement, such as adjusting the view based on mouse movement, scroll wheel input, and keyboard arrow keys. This makes the scene adaptable to a wide range of user needs, ensuring that dynamic changes in viewpoint are immediately responsive to user interaction.
- **ZoomControl():** This function manages zooming in and out of the 3D scene based on mouse scroll. It calculates the new zoom factor and adjusts the camera's field of view to provide a smooth zooming effect. By keeping the zoom functionality separate, this function enhances modularity and allows for easy adjustments without affecting other camera controls. It can be reused in other projects that require similar zoom behaviors, providing a quick and efficient way to implement zooming without rewriting the logic each time.
- **ObjectSelection():** This function allows users to select objects in the scene using the mouse. It checks for intersections between the mouse pointer and objects in the scene by detecting when a user clicks on an object and interacts with it. If an object is selected, the

function highlights the object and allows the user to, for example, drag or scale it. This feature is essential for projects involving manipulation of objects in 3D space. It simplifies object selection and interaction without needing to rewrite the logic for each project. The function is designed to adapt easily to diverse types of user interaction and project requirements.

- **RenderScene():** This function is responsible for updating the screen with the status of the scene, including all objects and camera views. It handles cleaning up the screen, rendering objects, and applying shaders. By keeping the rendering logic separate, it enhances modularity and allows for easier debugging and optimization. This function can be used in any program that requires rendering in a 3D environment, ensuring that the core functionality of rendering is consistent and efficient. Its reusability across projects saves a lot of time and effort since the core rendering logic can be quickly adapted to suit different project requirements.